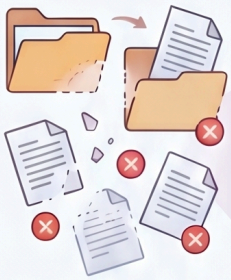
Research Infrastructure Guide
by Prof. Jeppe Rich (v0.2)

A Multi-Agent Framework for Resilient and Context-Aware Academic Research

Context Loss & Local Resilience Issues

Broken Context



Local Vulnerability

The Multi-Agent Framework & Two-Layer Architecture

AI Infrastructure (Workflow Layer)

Inside Agents: Claude Code / GPT

Session Logging & Context Loading

Manuscript Snapshots

Ensures Session History is Stored Locally

Wraps Local, Git-Tracked Infrastructure Around Tools

File Infrastructure

Powered by PowerShell Scripts (help)

Overleaf Syncing & Compilation

Versioning & Git-Tracked Safety Net

Ensures Manuscript Versions are Agent-Agnostic

The Five Key Pillars

Context-Aware Handover

Uses structured logs & HTML documents to orient new agents without re-briefing

Resilience & Local Safety Net

Local Git-tracked safety net for manuscript & code

Reviewer Loops

Autonomous manuscript revision

Directed Knowledge Graphs

Links projects via digests for efficient cross-project memory without high token costs

Proxy-Sandbox Isolation

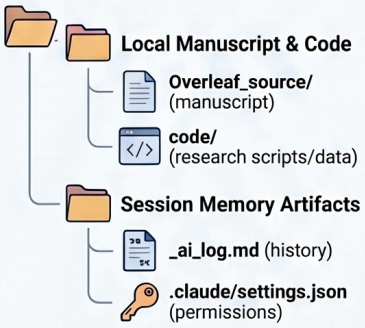
Structural privacy through organizational folder separation and path-based deny lists

- **Core Function:** Agent-to-agent orientation
- **Primary Tool:** **_ai_log.md / _handover.html**

- **Core Function:** Local Git-tracked safety net
- **Primary Tool:** **helpi 14** (snapshot)

- **Core Function:** Autonomous manuscript revision
- **Primary Tool:** **/respond [Round]**

Standard Project Folder Structure



Local Manuscript & Code

- **Overleaf_source/** (manuscript)
- **code/** (research scripts/data)

Session Memory Artifacts

- **_ai_log.md** (history)
- **.claude/settings.json** (permissions)

NotebookLM

Contents

Problem Formulation

A modern researcher writes papers in Overleaf, analyses data in Python, and increasingly relies on AI tools — Claude, GPT, NotebookLM — to accelerate every stage of the work. This is productive, but it creates a set of structural problems that none of those tools solve on their own.

i — Local file storage and resilience

Overleaf is convenient, but storing everything in a single cloud service is a real risk: outages, accidental overwrites, and loss of history with no local backup. The same applies to AI session context — if it lives only in a chat window, it is gone the moment the session ends. The infrastructure keeps a full local copy of every manuscript and every session log, git-tracked, on your own machine. Overleaf remains the collaboration layer; local storage is the safety net.

ii — AI infrastructure that leverages your paper knowledge

AI tools are only as useful as the context you give them. A generic AI session knows nothing about your prior work, your methods, or where a paper currently stands. Getting useful output requires either lengthy re-briefings or accepting shallow answers. This infrastructure solves that by keeping a structured session log (`_ai_log.md`) and a generated handover document (`_handover.html`) per project. Any AI agent — Claude, GPT, or a future tool — reads these files at session start and is immediately oriented: what the project is, what was done last time, and what comes next. No re-briefing. No lost context.

iii — Multi-agent, multi-skill integration

No single AI tool does everything well. Claude Code excels at code and structured edits; GPT-based tools have different strengths; NotebookLM handles literature differently again. Locking your workflow to one tool is fragile. This infrastructure is agent-agnostic: Claude Code and GPT Codex CLI both run the same session commands (`/work` , `/close` , `/snapshot` , `/family`), write to the same log format, and read the same handover files. Switching agents mid-project, or using different agents for different tasks, requires no extra setup. The handover document is the universal hand-off mechanism.

iv — Cross-project knowledge without full RAG

With 50+ active papers, knowledge compounds across projects: a method developed in one paper is relevant to three others; a dataset used in an old project reappears in a new one. A full vector RAG pipeline (embeddings, chunking, retrieval, staleness management) would handle this mechanically but at significant complexity and token cost, and it removes you from the loop. Instead, this infrastructure uses a directed digest model: you explicitly register which projects inform which others (`/family`), Claude or GPT reads those projects once and distils a compact structured

digest, and from then on only new sessions are read as deltas. The result is a many-to-many knowledge graph across your portfolio, at a fraction of the token cost — and because you control the links, expert knowledge is applied precisely where it is relevant, not retrieved by similarity from a black-box index.

v — Convergence Forum: Multi-agent and Single-agent debate

Complex research decisions (methodology choice, proof verification, gap detection) often benefit from adversarial debate. The **Convergence Forum** (`helpi 25`) orchestrates a discussion where agents challenge each other's assumptions. It supports two modes:

- **Forum (default):** Multi-agent round-robin (Claude → Gemini → Codex).
- **SAD (Single-Agent Debate):** A single agent takes three distinct roles sequentially: **The Critic** (skeptical peer reviewer), **The Advocate** (visionary co-author), and **The Realist** (Senior PI / Auditor). This mode is highly effective at breaking AI sycophancy with lower token overhead.

The forum supports five built-in **Consensus Templates** (task shortcuts) for standardized research auditing:

- `lit-review` : Systematic search and gap detection matrix builder.
- `math-verify` : Formal proof auditor (Assumption check, Notational integrity).
- `repro-audit` : 5-check verifier for reproduction suites (Seeds, Artifact parity).
- `final-pass` : 7-pillar manuscript audit (Consistency, Seams, Compliance).
- `code-audit` : PowerShell syntax verification and architectural opinion.

The `-Stage` parameter controls how conservative agents are — calibrated to the manuscript lifecycle:

- **draft** (default): Full adversarial stress-testing; agents may suggest major changes.
- **revision**: Surgical edits only — use when the paper is in an R1/R2 revision cycle.
- **final**: Defect detection only — no suggestions, no restructuring; use before submission.

To prevent token bloat, both modes use a **Blackboard Model**: the next agent turn sees `forum_state.md` , not the full transcript. Full prompts and outputs are saved under `_forums/YYYY-MM-DD_HH-MM-SS/` , while the moderator updates a compact state with four durable sections: Convergence Log, Active Arena, Parking Lot, and Latest Digests. The script validates that structure before accepting moderator rewrites, mirrors settled decisions into `convergence_log.md` , and stops explicitly on `Status: converged` , `adjourned` , or `failed` .

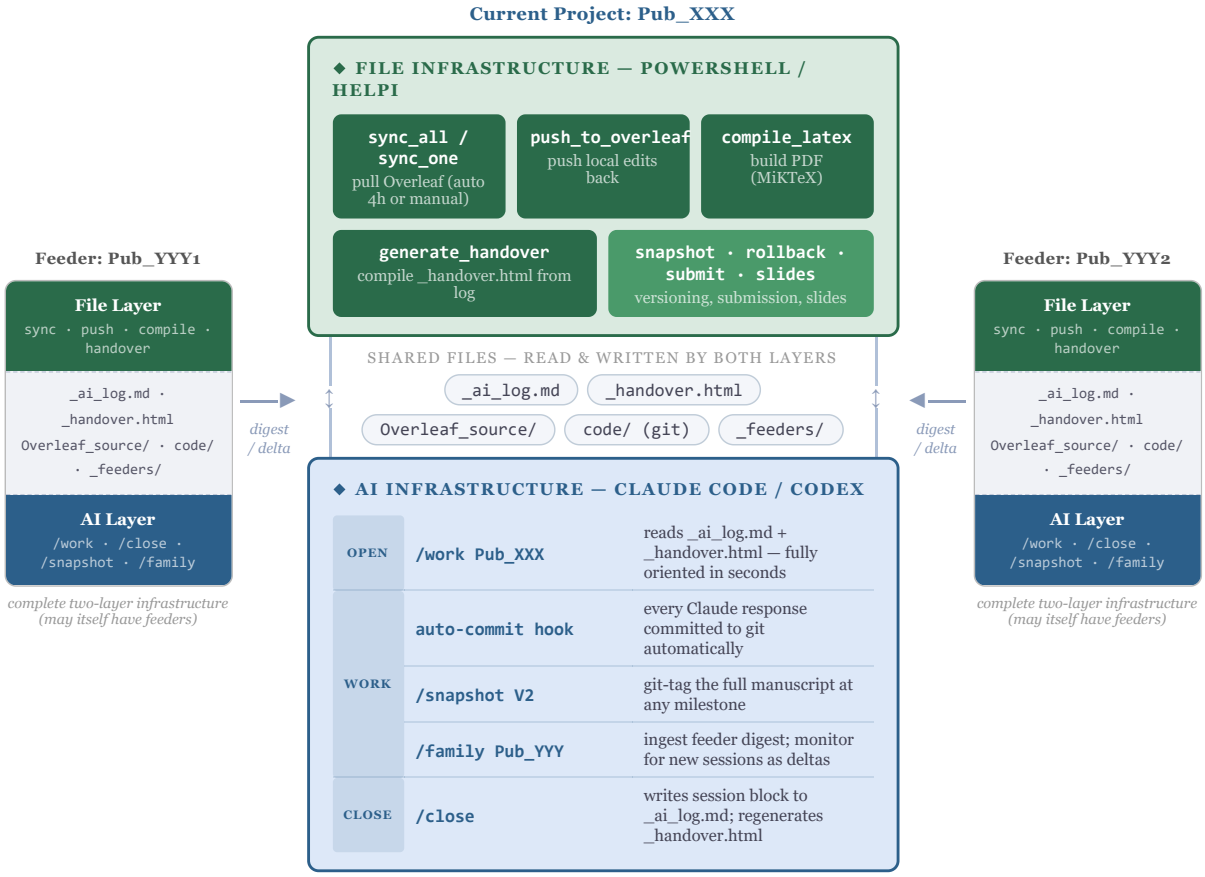
vi — Speed, ease of use, and openness

The infrastructure does not replace your existing tools — it wraps around them. You still write in Overleaf. You still use NotebookLM for literature synthesis. You still run analyses however you choose. What the infrastructure adds is: every edit is tracked, every AI session is logged, every version is recoverable, and any agent can pick up exactly where the last one left off. The overhead is minimal — a `helpi` command to push, a `/work` to open, a `/close` to log. Everything else is automatic.

Overview — Two Complementary Infrastructures

The research setup is split into two layers that are tightly coupled through a shared set of files. The **File Infrastructure** handles all mechanical operations — syncing with Overleaf, version control, LaTeX compilation, and folder scaffolding — and lives entirely in PowerShell scripts invoked via `helpi` . The **AI Infrastructure** is the workflow layer that runs *inside* Claude Code (or GPT): it loads project context, manages session logging, and creates manuscript snapshots.

Crucially, every project is an identical instance of this same two-layer stack. When one project draws on knowledge from another, the feeder is not a passive file dump — it is a full peer with its own Overleaf sync, git history, session log, and AI commands. The `/family` command reads a feeder's living infrastructure (handover + log) and distils it into a compact digest. From then on, `/work` monitors only the delta, giving you a many-to-many knowledge graph across projects at very low ongoing token cost.



Each feeder is a full instance of the same two-layer infrastructure and may itself have feeders — forming a many-to-many knowledge graph, not a simple hierarchy.

Project folder structure

Each paper lives under `JR\Publikationer\Pub_<Name>\` :

- `Overleaf_source/` — LaTeX manuscript, git-cloned from Overleaf
- `code/` — Python research code, git-tracked locally
- `code/data/` — Raw and processed data
- `Literature/` — PDFs and references
- `_ai_log.md` — AI session log (read and written by both layers)
- `_handover.html` — Generated handover document for agent hand-off
- `.claude/settings.json` — Per-project Claude permissions (proxy-sandbox, see §7b)

PART 1 — FILE INFRASTRUCTURE (POWERSHELL / HELPI)

1 — Create New Project ONCE per project

Scaffolds the full project folder structure, initialises git, registers the project in `projects.json`, creates a blank `_ai_log.md`, and sets up the per-project `.claude/settings.json` (proxy-sandbox deny rules). Optionally clones an existing Overleaf git URL.

```
helpi 1 Pub_MyNewPaper_TBA
helpi 1 Pub_MyNewPaper_TBA -GitUrl https://git.overleaf.com/<project-id>
```

What it creates

```
Pub_MyNewPaper_TBA/
  Overleaf_source/          # cloned from Overleaf (or empty if no -GitUrl)
  code/                     # Python/R code — git-initialised
```

```
    data/                  # anonymised data goes here
    .gitignore
  Literature/              # PDFs and references
  _ai_log.md               # blank session log — first entry on /work
  .claude/
    CLAUDE.md              # project brief — auto-filled by Claude on first /work
    settings.json          # proxy-sandbox deny rules (auto-generated)
```

After running, open the project with `helpi 5 Pub_XXX` then `/work Pub_XXX` in Claude Code — Claude reads the empty log and fills in `.claude/CLAUDE.md` from the manuscript.

> **Registry:** All projects are tracked in `AI_auto\projects.json`. The auto-sync task (helpi 2) reads this registry to know which projects to pull every 4 hours. New projects are added automatically on creation.

2/3 — Overleaf Sync AUTO every 4h

Every Overleaf project is cloned as a standard git repository into its `Overleaf_source/` folder. A Windows Task Scheduler job runs `sync_all.ps1` every 4 hours to pull updates from all registered projects.

How it works

- Checks the remote HEAD SHA via `git ls-remote` — skips the pull entirely if nothing changed on Overleaf
- Pulls all ~90 repos **in parallel** (20 threads) — total time ~30–60 s
- Auto-commits any local uncommitted changes with an `Auto-backup HH:MM` message
- Logs every result to `AI_auto\logs\sync_YYYY-MM-DD.log`

Registry

All registered projects are listed in `AI_auto\projects.json`. New projects added to `papers.csv` are cloned and registered automatically on the next sync run.

Manual trigger

```
powershell -File "C:\...\AI_auto\scripts\sync_all.ps1"
```

4 — Push to Overleaf MANUAL

When you edit `Overleaf_source/` files locally (e.g. after a Claude session), push them back to Overleaf with:

```
.\AI_auto\scripts\push_to_overleaf.ps1 -Project Pub_AssesTiming_Raoul_TBA
```

The script stages all changes, shows a summary of what will be pushed, commits with a timestamped message, then **pulls before pushing**:

1. Fetches the remote to check for new commits (e.g. from a co-author on Overleaf)
2. If the remote is ahead, rebases your local commits on top of theirs — so your changes land after theirs in history, cleanly
3. Pushes to `origin` (Overleaf's git endpoint). Overleaf picks up the changes immediately.

If the same lines were edited by both sides, the rebase stops and places conflict markers in the affected `.tex` file:

```
<<<<<<< HEAD
your version of the sentence
=======
co-author's version of the sentence
>>>>>>> origin/master
```

Edit the file to keep the right version, remove the markers, then run:

```
git -C "...\Pub_XXX\Overleaf_source" rebase --continue
```

and push again. This is the only scenario that requires manual intervention — it means you and a co-author edited the exact same lines at the same time.

> **Note:** You can also run this from inside any `Overleaf_source/` folder without the `-Project` argument.

5/6 — Compile & Open (MiKTeX + VS Code)

Installation

- **MiKTeX 25.12** — installed at `AppData\Local\Programs\MiKTeX\`
- **VS Code 1.114** with **LaTeX Workshop** extension (James Yu)
- MiKTeX set to **auto-install missing packages** silently on first use

VS Code workflow (recommended)

1. Open `Overleaf_source/` in VS Code: `code "...\Pub_XXX\Overleaf_source"`
2. Open the main `.tex` file
3. It compiles **automatically on every save** — PDF appears in a side tab
4. **Ctrl+Alt+J** — jump from source to PDF location (forward search)
5. Click in the PDF — jumps to the matching source line (inverse search)

Build artefacts go into `Overleaf_source/out/` and are cleaned after each successful build.

CLI compile

```
.\AI_auto\scripts\compile_latex.ps1 -Project Pub_AssesTiming_Raoul_TBA
```

Auto-detects `main.tex`, compiles with `latexmk` (handles bibliography passes), and opens the PDF in your default viewer.

7 — Session Logging & Handover

Session log — `_ai_log.md`

Each project root contains `_ai_log.md` — the living record of all AI sessions. Claude writes a new `## Session YYYY-MM-DD` block at the start and end of every working session via the `/work` and `/close` commands. Any agent (Claude, GPT, etc.) can read and update this file.

```
## Session 2026-04-04
**Agent:** Claude Sonnet 4.6
**Goal:** Refactor surrogate model in code/surrogate.py
**Files touched:**
- `code/surrogate.py` -- replaced polynomial fit with Gaussian process
**Outcome:** GP surrogate implemented, RMSE reduced from 0.12 to 0.04
**Next steps:** Run full PMIP sweep with new surrogate
**Git ref:** a3f9c12
```

Log compression *AUTO on /close*

As a project matures, `_ai_log.md` grows and starts consuming tokens on every session start. `compress_log.ps1` (called automatically by `/close` via `helpi 22`) keeps the log lean with three tiers: the **last 4 sessions verbatim** (active tier), older sessions compressed to **one-liners** under `## Compressed sessions`, and entries beyond 16 one-liners **archived** to `_ai_log_archive.md`. The handover is always compiled from current state, so it stays focused on recent activity.

```
helpi 22 Pub_XXX    # compress on demand; also runs automatically at /close
```

Auto-commit hook AUTO

A Claude Code *Stop hook* runs `auto_commit_hook.ps1` after every Claude response. It walks up from the current directory, finds the nearest git repo, and commits any file changes with `claude: auto-commit HH:MM`. This gives rollback granularity at the turn level.

Incremental session draft AUTO

A *PostToolUse hook* fires after every Edit, Write, or Bash tool call and appends a timestamped line to `_session_draft.md` in the project root. This gives a real-time, crash-safe record of what happened during the session — independent of whether `/close` is run. At `/close`, the draft becomes the authoritative file-touch manifest for the session block, then is deleted.

```
[2026-04-24 09:15] Edit:  Overleaf_source/main.tex
[2026-04-24 09:22] Bash:  helpi 2 Pub_XXX
[2026-04-24 09:31] Write: code/model.R
```

If a session ends without `/close`, the draft survives and is picked up next time. Nothing is lost.

Handover document AUTO

`helpi 7` closes the session, regenerates the handover, and opens the browser in one step. The source of truth remains `_ai_log.md`; the command runs `claude -p close.md`, then `generate_handover.ps1`, emitting both `_handover.html` and a machine-readable `_handover.json` sidecar. The handover also regenerates automatically every 5 minutes via the auto-handover scheduled task (if `_ai_log.md` has changed). You can also rebuild manually at any time:

```
helpi 7 Pub_AssesTiming_Raoul_TBA
```

The handoff package contains:

- A latest-session snapshot (goal, outcome, next steps, files touched, git ref when present)
- Validation of whether the newest session block is complete enough for handoff
- Instructions for any AI agent on how to add a session entry
- The rendered session log
- A copyable textarea with the raw `_ai_log.md` — paste to GPT, get updated markdown back, paste into `_ai_log.md`, regenerate HTML
- Git history for both `code/` and `Overleaf_source/`
- A structured `_handover.json` sidecar for future automation

AGENTS.md — automatic agent context file AUTO on /close

`helpi 7` also writes `AGENTS.md` to the project root. This file is auto-loaded by Codex CLI (and any agent following the `AGENTS.md` convention) when it starts in the project directory — the direct equivalent of how Claude Code reads `.claude/CLAUDE.md` automatically. It contains:

- Paper description (from `.claude/CLAUDE.md`)
- Latest session snapshot — goal, outcome, next steps
- Full session log (compact enough to include verbatim)
- Instructions for adding a session entry

Switching agents in either direction — Claude to Codex or Codex to Claude — requires no manual handover step. Both agents read the same files, write to the same `_ai_log.md`, and regenerate `AGENTS.md` on every `/close`. The context is always current.

Collaborator handover via Overleaf AUTO on /close

At `/close`, a compact `_handover_JR.md` is written into `Overleaf_source/`. The next `helpi 2` push carries it to Overleaf alongside the manuscript. When a collaborator who also uses the infrastructure runs `/work`, their session start automatically scans for peer handover files and surfaces them:

```
> Collaborator handover found: _handover_MN.md (2026-04-23)
  Goal: Add robustness checks — Outcome: Done — Next steps: Check Table 3
```

Collaborators without the infrastructure simply see extra files in Overleaf and can ignore them. No friction either way.

Platform map auto-update *AUTO on /close*

At `/close`, Claude scans the session for new platform or environment discoveries (tool failures, missing software, path issues) and cross-checks against the known-issues list. If a genuinely new, reusable pattern is found, it is added silently to both the inline *Platform facts* block in `~/.claude/CLAUDE.md` and the full reference in `AI_auto/known_issues.md`. At most 1–2 entries per session; one-off flukes are ignored.

One-time project setup *ONCE*

```
.\AI_auto\scripts\init_project_git.ps1 -Project Pub_AssesTiming_Raoul_TBA
```

Initialises a git repo in `code/`, adds a Python `.gitignore`, makes the first commit, and creates a blank `_ai_log.md`.

8 — Snapshot MANUAL & 9 — Rollback

Snapshot (helpi 8) — freeze a named version

Git-tags the complete `Overleaf_source/` state (all `.tex`, `.bib`, figures, and style files). Use before any major rewrite or before submitting. Also callable via `/snapshot` inside Claude Code.

```
helpi 8 Pub_XXX            # auto-increments version tag
helpi 8 Pub_XXX -Tag V2   # explicit tag: snapshot-v2
```

```
git tag -l "snapshot-*"                   # list all snapshots
git diff snapshot-v2 HEAD                 # what changed since V2
git checkout snapshot-v2 -- main.tex      # restore one file from snapshot
```

Rollback (helpi 9) — undo last N commits

Previews the diff before reverting. Every Claude turn is auto-committed (Stop hook), so rollback is fine-grained down to individual responses.

```
helpi 9 Pub_XXX -N 1      # preview + undo last commit
helpi 9 Pub_XXX -N 3      # undo last 3 commits
```

10 — Submit & 11/12 — Reviewer Loop

helpi 10 — Build submission package

Assembles a complete, journal-ready submission package in `_submissions/YYYY-MM-DD_<journal>/`. Run `/submit` in Claude Code first to generate cover letter, highlights, and author statement; `helpi 10` then runs the 7-step file assembly (compile, anonymise, diff, zip). Full details in Section B.

```
helpi 10 Pub_XXX
```

helpi 11 — Reviewer scaffold (Step 1)

Reads the reviewer `.txt` file, generates a `Response_R1.tex` scaffold with numbered `\TODO{}` slots for every comment, and pushes to Overleaf. Run once when reviewer comments arrive.

```
helpi 11 Pub_XXX
```

helpi 12 — Reviewer draft loop (Step 2)

Pulls the latest scaffold from Overleaf, drafts all `\TODO{}` slots using manuscript context, and pushes the completed draft back. Runs overnight autonomously via `/respond` in Claude Code. Full details in Section C.

```
helpi 12 Pub_XXX
```

13–25 — Status, Network, Guide, Docs, Cheatsheet, Model-check, Slides, Compress Log, Push GitHub, Technical One-pager, Convergence Forum

#	What it does	Usage
13	Project status dashboard — shows all projects, uncommitted changes, Overleaf sync state, last session date	`helpi 13`
14	Open project network graph — interactive HTML graph of all projects and feeder links with cluster shading. See Section D for the live graph.	`helpi 14`
15	Open this infrastructure guide in the browser	`helpi 15`
16	Regenerate infrastructure docs — rebuilds `infrastructure_summary.html/pdf` and `infrastructure_full.html/pdf` from source	`helpi 16`
17	Claude Code in-session cheatsheet — prints all slash commands, custom skills, model IDs, and tips. Also callable as `/helpi 17` from inside a Claude session.	`helpi 17`
18	Toggle Claude model-check on/off — enables or disables Claude suggesting Haiku for simple tasks. Edits the memory file; persists to the next session. Tell Claude "model check off" verbally for the current session only.	`helpi 18`
19	Generate Beamer slides from paper — safety-pulls from Overleaf first, then asks duration (12/30/45 min), depth (overview/standard/deep-dive), audience (conference/research-group/non-specialist), emphasis (balanced/methods-heavy/results-heavy). Writes `slides_main.tex` into `Overleaf_source/` and offers to push. Three presets skip the questions: `quick`, `seminar`, `public`.	`helpi 19 Pub_XXX [preset]`
20	Restore infrastructure on a replacement machine — checks Claude CLI, Git, MiKTeX, VS Code, PowerShell profile, `~/.claude/` folder, scheduled task, and `projects.json`. Fixes what it can automatically; reports items requiring manual action.	`helpi 20`
21	First-time setup wizard for a new user or machine — guides through: publications root, git identity, tool path detection, writes `config.ps1`, wires the `helpi` function into the PowerShell profile, and registers the auto-sync scheduled task.	`helpi 21`
22	Compress AI log — trims `_ai_log.md` to the last 4 sessions verbatim; older sessions are collapsed to one-liners under a `## Compressed sessions` header; entries beyond 16 are archived to `_ai_log_archive.md`. Runs automatically at `/close`; call manually to compress on demand.	`helpi 22 Pub_XXX`
23	Push code/ to GitHub — pushes the project's `code/` git repo to GitHub. If no remote exists, creates the GitHub repo via `gh` CLI (no browser needed), sets it as `origin`, and pushes in one step. Default repo name: `<project>-code` (private). Pass a custom name as third argument; `public` as fourth for a public repo. Requires `gh auth login`.	`helpi 23 Pub_XXX`
24	Technical one-pager from paper — reads the main manuscript and generates a self-contained LaTeX note distilling the core idea, key assumptions, main result, and key formulas onto a single A4 page. Compiles to PDF automatically. Useful before a talk, a referee response, or a project kickoff to refresh the central ideas quickly.	`helpi 24 Pub_XXX`
25	Convergence Forum — adversarial multi-agent debate to reach a stress-tested consensus on a research task. Two modes: **Forum** (Claude, Gemini, Codex challenge each other in structured rounds with a Blackboard); **SAD** (single agent in three roles: Critic, Advocate, Realist). `-Stage` controls conservatism: *draft* (full open debate), *revision* (surgical edits only), *final* (defect detection only). Output: structured Markdown report with consensus position and dissenting views.	`helpi 25 Pub_XXX 'Task description' [-Mode SAD] [-Stage draft|revision|final]`

Quick Command Reference — `helpi 1–25`

Calling conventions

```
helpi                           # show all commands (project auto-detected from CWD)
helpi 3                         # run command 3 (uses detected project, or prompts)
helpi 3 Pub_AssesTiming_Raoul_TBA  # run command 3 for a specific project
```

```
helpi compi                       # partial name match → resolves to command 3
helpi 3 Pub_X main.tex            # run command 3 with a specific .tex file
```

Context awareness — CWD project detection

`helpi` automatically detects which project you are in by reading your current working directory. If any folder segment in your path matches a known project name (e.g. `Pub_`, `Pro_`, `PhD_` prefixes), that project is pre-filled into every example command in the list — no need to type it. Works from any subfolder (`code/`, `Overleaf_source/`, etc.).

Partial name matching

You can type a fragment of any command name instead of a number. `helpi compi` matches *Compile LaTeX + open PDF* (the only command starting with "compi") and runs it directly. If your fragment matches more than one command, helpi lists the candidates so you can pick by number.

Preview & confirm before every run

Every `helpi <N>` invocation shows the exact underlying script line before executing, e.g.:

```
[3] Compile LaTeX + open PDF  ->  Pub_AssesTiming_Raoul_TBA
compile_latex.ps1 -Project Pub_AssesTiming_Raoul_TBA

[Enter] run  |  [any other key] cancel  (press Up to edit at prompt)
```

- **Enter** — runs the command as shown.
- **Any other key** — cancels. The command is pre-loaded into shell history: press ↑ at the next prompt to retrieve it, edit freely, then press Enter.

From inside Claude Code — the confirmation prompt is automatically skipped when `helpi` detects a non-interactive context (stdin redirected). Running `! helpi 13` from the Claude prompt executes immediately with no keypress required. The `-Force` flag achieves the same effect explicitly.

Tab completion

Registered in PowerShell profile:

- `helpi compi<Tab>` — completes to `helpi 3`; dropdown shows `[3] Compile LaTeX + open PDF`; tooltip shows the actual script line with the detected project filled in.
- `helpi 3 <Tab>` — completes the project name from `projects.json`.
- `helpi 3 Pub_X <Tab>` — lists `.tex` files in `Overleaf_source/`, sorted most-recently-modified first; tooltip shows last-modified timestamp.
- Same `-TexFile` completion is also available directly on `compile_latex`.

#	What it does	Command	Type
1	Create new project	`new_project.ps1 -Project Pub_XXX [-GitUrl url]` — scaffolds folders, inits git, creates `_ai_log.md`; auto-registers auto-handover task	once
2	Pull all projects from Overleaf	`sync_all.ps1`	auto 4h
3	Pull one project from Overleaf	`sync_one.ps1 -Project Pub_XXX` — instant pull without waiting for 4h scheduler	manual
4	Push local edits to Overleaf	`push_to_overleaf.ps1 -Project Pub_XXX`	manual
5	Compile + open project (VS Code + PDF)	`open_project.ps1 -Project Pub_XXX` — compiles first, then opens VS Code + Explorer + PDF	manual
6	Compile LaTeX + open PDF	`compile_latex.ps1 -Project Pub_XXX` — graphical .tex picker if multiple files	manual
7	Log + handover + open in browser	`claude -p close.md` then `generate_handover.ps1` then opens `_handover.html` — one command to close session and refresh handover	manual
8	Snapshot Overleaf source (git tag)	`snapshot.ps1 -Project Pub_XXX [-Tag V2]` — git-tags full Overleaf_source/ (.tex, .bib, figures); auto-increments version if Tag omitted	manual
9	Rollback last N code commits	`rollback.ps1 -Project Pub_XXX -N 1` — previews changes before reverting	manual

10	Build submission package	`submit.ps1 -Project Pub_XXX` — prompts for journal, main .tex, and original .tex (for diff); assembles complete package in `_submissions/YYYY-MM-DD_journal/` . Run `/submit` first to also generate cover letter, highlights, and author statement.	manual
11	Reviewer scaffold (.txt → LaTeX + push)	Reads reviewer `.txt` file, generates `Response_R1.tex` scaffold with `\TODO{}` slots, pushes to Overleaf — Step 1 of two-step review loop	manual
12	Reviewer draft loop (pull → draft → push)	Pulls latest from Overleaf, drafts all `\TODO{}` slots using manuscript context, pushes back — Step 2 of two-step review loop	manual
13	Project status dashboard	`status.ps1` — shows active projects, uncommitted changes, Overleaf sync state	manual
14	Open project network graph	`network.ps1` — generates and opens interactive HTML graph of all projects and feeder links, with cluster shading	manual
15	Open infrastructure guide	`Start-Process "AI_auto\infrastructure.html"`	manual
16	Generate docs (summary + full HTML/PDF)	`generate_docs.ps1` — rebuilds `infrastructure_summary.html/pdf` and `infrastructure_full.html/pdf`	manual
17	Claude Code in-session command reference	Prints a cheatsheet of all slash commands, custom skills, model IDs, and keyboard tips. Also callable as `/helpi 17` from inside a Claude session.	manual
18	Toggle Claude model-check on/off	Enables or disables the behavior where Claude suggests switching to Haiku for simple tasks. Edits the memory file -- persists to the next session. For the current session, tell Claude "model check off" verbally.	manual
19	Generate Beamer slides from paper	Pulls from Overleaf (safety net), then asks: duration (12/30/45 min), depth (overview/standard/deep-dive), audience (conference/research-group/non-specialist), emphasis (balanced/methods-heavy/results-heavy). Writes `slides_main.tex` into `Overleaf_source/` and offers to push. Regeneration overwrites slides -- manual Overleaf edits survive only if you pull before regenerating.	manual
20	Restore infrastructure on replacement machine	`restore.ps1` — checks Claude CLI, Git, MiKTeX, VS Code, PowerShell profile, `~/.claude/` folder, scheduled task, and `projects.json` ; fixes what it can automatically	once
21	First-time setup wizard (new user / new machine)	`setup.ps1` — interactive wizard: sets publication root, git identity, tool paths, writes `config.ps1` , wires `helpi` into PowerShell profile, registers auto-sync task	once
22	Compress AI log (trim old sessions)	`compress_log.ps1 -Project Pub_XXX` — keeps last 4 sessions verbatim; collapses older ones to one-liners; archives beyond 16. Runs automatically at `/close` .	manual
23	Push code/ to GitHub	`push_to_github.ps1 -Project Pub_XXX` — creates GitHub repo via `gh` CLI if no remote exists, then pushes. No browser needed. Default: private repo named `<project>-code` .	manual
24	Boil paper to technical one-pager	Reads main .tex and generates a self-contained LaTeX note distilling core ideas, formulas, and checks. Compiles to one A4 page. Uses `generate_onepager.ps1` .	manual
25	Convergence Forum (Multi-agent / SAD debate)	`run_forum.ps1 -Project XXX -Task '...' [-Mode Forum|SAD] [-Stage draft|revision|final]` — orchestrates adversarial debate to reach stress-tested consensus. **Forum** mode uses multiple agents (default: Claude, Gemini, Codex); **SAD** mode uses one agent in three roles (Critic, Advocate, Realist). `-Stage` controls agent conservatism: **draft** (full debate), **revision** (surgical edits), **final** (defect detection only).	manual

PART 2 — AI INFRASTRUCTURE (CLAUDE CODE & AI COMMANDS, A–G)

A — Claude Code Commands

These are slash commands registered in `~\.claude\commands\` and are available in any Claude Code session. They handle the three pivotal moments of an AI working session: opening, checkpointing, and closing.

Global behaviour is set in `~\.claude\CLAUDE.md` : if you paste a bare project path into the Claude prompt with no other instruction, Claude automatically treats it as `/work <path>` .

/work — open a project session AI

Reads `_ai_log.md` and `_handover.html`, summarises the last session and any open next-steps, checks git status of `code/`, then asks for today's goal before writing anything to the log.

```
/work Pub_AssesTiming_Raoul_TBA

# or paste the full path — both are equivalent:
C:\Users\rich\OneDrive - Danmarks Tekniske Universitet\JR\Publikationer\Pub_AssesTiming_Raoul_TBA
```

/snapshot — version the full manuscript AI

Creates an annotated **git tag** on `Overleaf_source/`, capturing the complete state: all `.tex` files, `.bib` bibliography, figures, style files, and subdirectories. Auto-increments the version number if omitted. Calls `snapshot.ps1` under the hood (also available as `helpi 8`).

```
/snapshot V2
/snapshot                         # auto-detects next version
/snapshot V2 before-reviewer-cuts  # tag: snapshot-v2-before-reviewer-cuts
```

Useful git commands after snapshotting:

```
git tag -l "snapshot-*"                    # list all snapshots
git diff snapshot-v2 HEAD                  # what changed since V2
git checkout snapshot-v2 -- main.tex       # restore one file
git checkout snapshot-v2                   # restore everything
```

/close — end a session AI

Appends the closing block to `_ai_log.md` (files touched, outcome, next steps, git ref), then automatically:

```
/close
```

- **Updates** `.claude/CLAUDE.md` — if the session changed the project phase, locked new notation, froze a file, or created a new manuscript version, Claude edits the relevant sections and reports what changed (or "CLAUDE.md unchanged")
- **Regenerates** `_handover.html` — run `helpi 7 Pub_XXX` to close session, regenerate handover, and open it in browser in one step
- Reminds you to push to Overleaf if manuscript files were changed: `helpi 4 Pub_XXX`

Command	Argument	What it does
`/work`	project name or full path (optional if CWD is inside a project)	Load context from log + handover, summarise, ask for goal, write session-start block
`/snapshot`	`V2` or empty (auto) or `V2 label`	Git-tag full Overleaf_source/ as a named frozen checkpoint
`/close`	—	Complete session block in _ai_log.md; auto-update .claude/CLAUDE.md if project state changed; auto-regenerate _handover.html
`/helpi [N] [proj]`	command number + optional project (auto-detected from CWD if omitted)	Run any helpi command without leaving Claude. Skips the confirmation prompt automatically. `/helpi` alone shows the full menu.
`/shell`	—	Enter shell passthrough mode — every subsequent message is executed as a shell command. Type `exit` or `done` to return to normal conversation.
`/respond`	`R1` or `R1 --draft response_R1.tex`	Autonomous reviewer response loop — see Section C below
`/submit`	empty (first submission) or `R1`, `R2`	Generate complete submission package — see Section B below
`/grill-paper`	—	Pre-submission stress-test — AI reads manuscript, grills author across contribution, rigor, and weaknesses. Dual grade (paper + author, 0–10) per question. Logs all issues to `Overleaf_source/grill_log.md`. No edits during session — see Section E below.

`/grill-edit`	—	Edit phase after `/grill-paper` — works through `grill_log.md` issue by issue, proposes concrete edits, records author decisions (approve / adjust / dismiss with reason), updates submission verdict to READY when all blocking issues resolved.

Writing style — reference papers AI

All agents apply a consistent academic writing style when editing or drafting research text. The style is derived from five canonical papers stored in:

```
AI_auto\literature\Reference_Papers\
```

Paper	Why it is here
Einstein (1905) — On the Electrodynamics of Moving Bodies	Economy, concrete examples before abstraction, long sentences that carry exactly one logical load
Akerlof (1970) — The Market for Lemons	Plain prose for abstract theory, inevitable logical flow, no inflation
Kahneman & Tversky (1979) — Prospect Theory	Precise, direct, each claim immediately supported; no hedging
Ioannidis (2005) — Why Most Published Research Findings Are False	Bold claims stated plainly, structured reasoning, confident without arrogance
Dantzig & Thapa — Linear Programming: Introduction	Mathematical exposition that is precise without being opaque

The full style guide (`STYLE_GUIDE.md` in the same folder) is wired into `~\.claude\CLAUDE.md` (Claude) and injected into `AGENTS.md` on every `helpi 7` regeneration (Codex, Gemini). All agents apply the same rules without being told each session.

Core rules: economy (cut every word that adds no force); concrete before abstract; active voice and first person plural; transitions that carry logical force (*This implies, It follows that*); one logical load per sentence; consistent terminology with no synonym rotation. A banned-phrase list eliminates common AI-isms: *delve into, showcase, leverage, nuanced, it is worth noting that, in order to, utilize, furthermore*, and similar.

/style-edit — Background prose style editor AI

`/style-edit` rewrites the prose of a LaTeX manuscript section-by-section to match the reference writing style. Math, equations, tables, figures, and all LaTeX commands are left completely untouched. Bibliography/references sections are always passed through verbatim. Output: a restyled `.tex` file, a `style_edit.diff`, and optionally a structured review document for change-by-change approval.

Flag	Default	Effect
`--project Pub_Name`	—	Resolves project path and auto-detects the main `.tex` file
`--agent claude|gemini|codex`	`claude`	Backend model. Use `gemini` or `codex` for long papers to bypass Claude rate limits
`--chunk-size N`	`4`	Sections per parallel job (gemini/codex only). A 20-section paper with chunk-size 4 launches 5 parallel jobs simultaneously
`--review`	off	After assembly, generate `style_review.md` listing every changed paragraph with `[KEEP]` / `[REJECT]` markers
`--out path`	`{project}\_style_edits\YYYY-MM-DD_HH-MM-SS\`	Override output directory

Parallel chunked processing (gemini/codex): sections are grouped into chunks of `--chunk-size` and each chunk is a separate parallel PowerShell job. A stalled job times out after 30 minutes and falls back to the original text for that chunk — the rest of the paper is unaffected. Token counts (in/out per chunk and totals) are written to `token_usage.tsv` and summarised in `style_edit_log.md`.

Typical usage:

```
/style-edit --project Pub_MyPaper --agent gemini --review
```

/style-apply — Selective change approval AI

After `/style-edit --review` produces `style_review.md`, open it and change `[KEEP]` to `[REJECT]` on any paragraph rewrite you want to discard. Then run:

```
/style-apply path/to/style_review.md
```

The skill starts from the fully-restyled file, reverts every `[REJECT]` back to the original text, and writes `[name]_approved.tex` plus `approved.diff`. Both open automatically. Unmarked changes default to `[KEEP]`. Safe to re-run after further edits to the review doc.

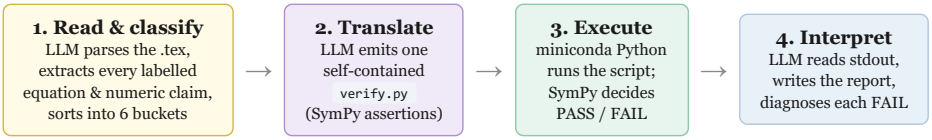
Step	Command
1. Run style edit with review	`/style-edit --project Pub_X --agent gemini --review`
2. Edit review doc	Open `style_review.md`, mark `[REJECT]` on unwanted changes, save
3. Apply approved changes	`/style-apply path/to/style_review.md`
4. Push to Overleaf	`helpi 4 Pub_X`

/verify-math — Machine-verifying the mathematics with SymPy AI

`/verify-math` answers a question every author has but rarely checks rigorously: *is the algebra in this paper actually correct?* An LLM (or a tired human) drops a term, flips a sign, mis-inverts a moment, or rounds a calibration the wrong way — silently, with full confidence. This skill does not *ask the model whether the math is right*; it **compiles each equation into an executable SymPy program and runs it**. The verdict comes from a computer algebra system, not from the model's own arithmetic. That distinction is the whole design.

The core idea: don't trust the LLM's arithmetic — make it write a program a CAS can run
LLMs are unreliable calculators. They are, however, excellent *translators* — turning a typeset equation into the equivalent `SymPy` expression is a language task, which is exactly what they are good at. The architecture exploits this split of labour: the model is used only to *read and translate*; all *judgement of truth* is delegated to SymPy's symbolic engine. The model never declares an equation correct. It writes code that, when executed, either reduces `LHS - RHS` to `0` or it does not. The author reads the machine's answer.

The four-stage pipeline: LaTeX → SymPy program → execution → interpretation



Stage 1 — Read & classify. The agent reads the source `.tex` in full and identifies every numbered/labelled equation (`equation`, `align`, `\[...\]`) and every explicit numeric claim in prose or tables. Each claim is keyed to its own `\label{}` (or an anchor like `Eq-near-"<quote>"`) so the final report speaks the author's own reference scheme. Each claim is then sorted into exactly one bucket:

Bucket	What it is	How SymPy checks it
`IDENTITY`	An equation asserting LHS = RHS	`simplify(expand(LHS - RHS)) == 0`
`SERIES`	A Taylor / Laurent / Mercator expansion	`series(expr, x, x0, n)`, compare coefficients
`SOLVE`	Inverting/solving for a variable (e.g. recovering σ^2 from CV)	`solve(Eq(...), var)`, compare to claimed closed form
`MOMENT`	A distributional moment (mean/variance/skewness/MGF)	`sympy.stats` or a verified closed form
`NUMERIC`	A concrete number computed from inputs (table cells, worked examples)	evaluate & compare to stated value, respecting rounding

`NOT-CHECKABLE`	Limit theorems (CLT, SLLN, Slutsky), existence/uniqueness, definitions, pure reasoning	— flagged with a one-line reason, never "proved"

Stage 2 — Translate to an executable program. The agent writes a single, self-contained `verify.py` covering every claim in the first five buckets. The translation is where the real engineering lives:

- **Assumptions are declared, not assumed.** Symbols are created with `positive=True`, `real=True` where the paper says so — these assumptions are exactly what let `simplify` close an identity that would otherwise stay unsimplified.
- **Symbolic-length sums are instantiated.** When an identity involves a sum to a symbolic upper limit (`J`, `N`, `K`) that will not simplify in general, the script evaluates it at concrete sizes (e.g. 2, 3, 4) and passes only if every instance gives `0` — and prints which sizes were used, so the scope of the check is explicit.
- **Every check is isolated.** Each assertion is wrapped in try/except, so one bad translation reports as `ERROR` rather than aborting the whole run.
- **Every check prints its own evidence:** one line per claim — `LABEL | BUCKET | PASS/FAIL | checked: <sympy expr> | got: <result> vs stated: <claim>`.

Stage 3 — Execute. The script is run with the miniconda Python interpreter where SymPy lives (`C:\Users\rich\AppData\Local\miniconda3\python.exe` — *not* on PATH, *not* the `pyopt` env). SymPy's symbolic engine, not the model, returns the verdict for each claim. If the script errors out wholesale, the agent fixes it and re-runs (a few attempts) before reporting — a script that won't run is a bug in the translation, not a result.

Stage 4 — Interpret. The agent reads the captured stdout and writes `math_check_report.md`: header totals (*N checked: X passed, Y failed, Z errored; W not-checkable*), a results table keyed to the paper's labels, a *Failures* section that, for each FAIL, shows the equation as written, its SymPy translation, the discrepancy, and the most likely cause (dropped term, sign error, wrong assumption, rounding, or a genuine translation ambiguity), and a *Not machine-checkable* section so the boundary of the audit is explicit.

Why the SymPy translation is shown for every check
The verification is only ever as good as the LaTeX→SymPy translation. A wrong formalization could pass a wrong equation. The skill defends against this by **printing the exact SymPy expression it checked alongside every verdict**. A green PASS that rests on a mis-translation is therefore *visible* — the author can read the "What was checked" column and catch a bad formalization, rather than having it hidden behind the result. The report carries an explicit caveat to that effect.

What it deliberately refuses to do
Convergence and limit-theorem arguments (CLT, SLLN, Slutsky, Jensen-as-applied), existence/uniqueness claims, and any step that is *mathematical reasoning rather than computation* are never marked PASS. They are listed as `NOT-CHECKABLE` with a one-line reason. The tool states precisely what it did *not* cover, so a clean run is never mistaken for a full proof check.

Flag	Default	Effect
`path/to/file.tex`	—	The LaTeX file to check (optional if `--project` is given)
`--project Pub_Name`	—	Resolves the project path and auto-detects the main `.tex` (prefers a file whose name contains `verif`)
`--out path`	`{project}\_math_checks\YYYY-MM-DD_HH-MM-SS\`	Override output directory
`--inline`	off (background agent)	Run directly in this session for a small file when you want the result immediately

Output (both open automatically when done): `math_check_report.md` — the audit, keyed to your equation labels — and `verify.py` — the self-contained, re-runnable SymPy script. A background run sends a `PushNotification` on completion with the pass/fail totals.

Typical usage:

```
/verify-math --project Pub_OptimismBias_PartA
/verify-math "C:\path\to\Math_Verification.tex"
/verify-math --project Pub_MyPaper --inline
```

Relationship to the Forum (helpi 25): this skill is the standalone counterpart to the Forum's verification pre-pass — the same translate-and-check logic can feed a "Verified facts" block into `helpi 25`, so multi-agent debate argues from machine-checked ground truth while staying read-only. The skill itself never edits the source manuscript; it only checks.

Agent task delegation — Haiku vs. Sonnet AI

Not every task warrants a frontier model. The infrastructure splits work by complexity to reduce both cost and latency.

Task type	Model	Ratio
Mechanical operations — session logging, file writes, log compression, handover regeneration, state card updates	Haiku	Formul no judg require Haiku i ~12× cheape 3–5× fa Runs fr short briefing prompt does nc reproce the full convers history
Research work — writing, editing, analysis, code review, proof checking, reviewer responses	Sonnet / Opus	Require context judgme and do knowle Frontie model quality matters here.

How /close uses this: when you run `/close`, the main Sonnet agent gathers session context (goal, outcome, files touched) and passes a compact briefing to a **Haiku subagent** via the `Agent` tool. Haiku performs all mechanical close operations (appending the log, compressing, writing the state card, regenerating the handover) without touching the conversation history. This makes each close faster and significantly cheaper, especially at the end of long sessions.

To switch models manually within a session, use `/model haiku` or `/model sonnet`. For the current session only — does not persist. Use `helpi 18` to toggle the permanent model-check behavior.

B — /submit — Submission package generator AI

Assembles a complete, journal-ready submission package in a timestamped subfolder `_submissions/YYYY-MM-DD_<journal>/` under the project root. Two entry points exist: `/submit` (AI + file ops) and `helpi 10` (auto-generates AI content via `claude -p` if staging is empty, then runs file ops).

Two-step flow

Step 1 — AI content: Claude reads the abstract and contributions, writes `cover_letter.tex`, `highlights.txt`, and `author_statement.tex` to `_submit_staging/`, and compiles them to PDF. `/submit` triggers this directly; `helpi 10` triggers it automatically via `claude -p` if `_submit_staging/cover_letter.pdf` is absent.

Step 2 — File assembly (`submit.ps1`): prompts for journal abbreviation, revised .tex, and original .tex (for diff), then runs all 7 steps below.

File naming convention

All outputs use a consistent prefix: `Type_Author_Journal_Year_Rev` (e.g. `Rich_MPC_2026_R1`). Revision is auto-detected from the .tex filename (e.g. `Springer_R1C.tex` → `R1`); override with `-Revision R2`.

Files produced

File	How generated	Action needed
`Manus_Rich_MPC_2026_R1.pdf`	Full latexmk compile (BIBINPUTS + BSTINPUTS set)	Verify layout
`Frontpage_Rich_MPC_2026_R1.pdf`	Page 1 via Ghostscript	Title-page upload
`Package_Rich_MPC_2026_R1.tex`	Flat .tex with .bbl inlined	Reference copy
`Package_Rich_MPC_2026_R1.zip`	Flat tex (as `main.tex`) + figures + .sty/.cls	Upload as source package
`Manus_Blind_MPC_2026_R1.pdf`	Author info stripped, compiled	Double-blind submissions
`Package_Blind_MPC_2026_R1.zip`	Blind tex (as `main.tex`) + figures	Double-blind source upload
`Diff_Rich_MPC_2026_R1.pdf`	latexdiff vs original (CRLF-normalised, bbl inlined)	Attach as tracked-changes
`Diff_Rich_MPC_2026_R1.tex`	Diff source for manual re-compile if needed	Open in Overleaf if PDF fails
`Cover_Rich_MPC_2026_R1.pdf`	Claude-drafted cover letter	Fill in journal name, review
`Highlights_Rich_MPC_2026_R1.txt`	5 bullets ≤ 85 chars	Paste into submission form
`AuthorStat_Rich_MPC_2026_R1.pdf`	CRediT taxonomy statement	Confirm roles
`MANIFEST.txt`	Contents summary with file sizes	Keep for records

Blind manuscript — what is stripped

A line-by-line state machine comments out (prefixes `% [BLIND]`) all author-identifying content: `\author{}`, `\institute{}`, `\affiliation{}`, `\address{}`, `\thanks{}`, `\email{}`, elsarticle markers (`\ead`, `\corref`, `\cortext`, `\fntext`), and the entire Acknowledgements section. Works for svjour3, elsarticle, and article classes.

Known gotchas & fixes implemented

- **BSTINPUTS not set** — bibtex could not find `.bst` files (e.g. `spmpsci.bst`) stored in `Overleaf_source/`, producing an empty .bbl. Fixed: `$env:BSTINPUTS` set alongside `$env:BIBINPUTS` before all compilations.
- **CRLF/LF mismatch in diff** — files saved on Windows (CRLF) vs Overleaf (LF) differ on every line; latexdiff cannot find anchors and silently drops changes. Fixed: LF-normalised temp copies are diffed; originals untouched.
- **PowerShell `-replace` corrupts .bbl** — `$` in bibliography entries (math in titles) is interpreted as a regex back-reference, corrupting or erasing the inlined .bbl. Fixed: `[regex]::Replace` with a MatchEvaluator used for all .bbl substitutions.
- **diff.tex compiled from wrong directory** — when compiled from the output folder, `svjour3.cls` and figure files are missing. Fixed: diff is compiled from `Overleaf_source/`; temp files cleaned up after.

Dependencies (pre-installed)

Tool	Location	Used for
Ghostscript 10.07	`C:\Program Files\gs\` (version-independent search)	Front-page extraction
latexdiff	MiKTeX scripts folder	Diff generation
Strawberry Perl	`C:\Strawberry\perl\bin\perl.exe`	Runs latexdiff (has Algorithm::Diff; Git Bash perl does not)
latexmk + MiKTeX	MiKTeX bin	Compilation (all 4 passes)

Note: pdftk is *not* required — Ghostscript handles front-page extraction. `spmpsci.bst` and other journal style files must be present in `Overleaf_source/` (they are downloaded automatically when you sync from Overleaf).

Usage

```
/submit                  # AI content + full package (recommended)
helpi 10                 # same — auto-calls claude -p if staging is empty, then submit.ps1
```

Prompts during submit.ps1

- Journal abbreviation (e.g. `MPC`, `TRB`) — folder name and prefix
- Original .tex for diff — graphical picker; cancel to skip diff
- Revised (main) .tex — graphical picker if multiple candidates
- Revision label — auto-detected from filename (`_R1C.tex` → `R1`); prompted only if undetectable

Output location

```
Pub_XXX/
  _submit_staging/          # staged AI content (.tex + .pdf); overwritten each run
  _submissions/
    2026-04-15_MPC/         # complete package — one folder per run, never overwritten
    2026-05-01_MPC/         # re-submission after edits
```

C — /respond — Autonomous reviewer response loop AI

Runs fully autonomously — launch before sleeping, wake up to a finished response letter. Handles comment parsing, manuscript edits, cross-reviewer clustering, and tone management without interruption. Flags only genuine conflicts and unresolvable comments for your attention.

File conventions (all in `Overleaf_source/`)

File	Role
`reviewers_R[n].txt`	All reviewer comments — paste the full text as received (one file, all reviewers)
`response_R[n].tex`	Response letter — generated by Claude, or your partial draft in continue mode
`review_log_R[n].md`	Tracking table — every comment, status, and manuscript change
`main.tex`	Working manuscript — Claude edits this directly
`snapshot-pre-R[n]`	Git tag taken automatically before the loop starts — full recovery point

Usage

```
/respond R1                           # clean start, round 1
/respond R1 --draft response_R1.tex   # continue from a partial draft
/respond R2                           # round 2 — reads reviewers_R2.txt
```

What the loop does

1. **Setup:** pulls from Overleaf, takes snapshot `snapshot-pre-R[n]`, reads inputs
2. **Parse:** numbers all comments R1.1, R1.2, R2.1 … identifies cross-cutting comments
3. **Pass 1:** addresses all Reviewer 1 comments — edits manuscript, drafts responses using title-based references
4. **Pass 2:** addresses Reviewer 2+ — cross-references R1 responses, flags reviewer conflicts
5. **Pass 3:** integration pass — unified cross-cutting responses, tone consistency, accuracy check
6. **Self-check:** evaluates whether another pass is needed; continues to Pass 4–5 if so (cap: 5 total)
7. **Structure freeze:** parses final `main.tex` section structure, resolves all title references to "Section 3.2 (Model Calibration)" format
8. **Diff:** runs `latexdiff` against `snapshot-pre-R[n]` to generate `manus_R[n]_diff.tex`
9. **Output:** writes `response_R[n].tex`, `manus_R[n]_diff.tex`, `review_log_R[n].md`, logs to `_ai_log.md`

Referencing convention

Throughout all passes, manuscript locations are referenced by **title only** — never by section number or line number. Section numbers shift every time a subsection is added, removed, or merged. Numbers are resolved in a single final pass after all edits are complete and the structure is frozen. Final references read: *"Section 3.2 (Model Calibration)"* — both number and title, so the reviewer can find the location even if their compiled version differs slightly.

Tone policy

- Always polite — even when pushing back
- Acknowledge when the reviewer is right, sincerely
- Push back with *"we respectfully disagree"* + a clear scientific argument when warranted
- Never dismissive; never sycophantic

Unresolved comments

Comments that cannot be addressed through editing (go deeper than the current revision allows) are marked **Fully unresolved** or **Partly unresolved** and collected in a dedicated section at the end of `response_R[n].tex` — rendered in red for immediate visibility. These are flagged in the final report for your decision.

Overnight workflow

```
# Before sleeping:
helpi 3 Pub_XXX          # pull latest (also done automatically inside /respond)
claude --full-auto       # launch Claude Code without approval pauses
/respond R1              # kick off the loop — leave terminal open
```

Trust: `C:\Users\rich\OneDrive - Danmarks Tekniske Universitet\JR` is already marked trusted in `~\.claude\` config. No extra permission setup needed. The blast radius is low — only LaTeX files in `Overleaf_source/` are touched, and the pre-revision snapshot is always available for rollback.

D — Cross-Project Memory — Feeder Projects

When a project builds on knowledge from earlier work, you can link those projects as **feeders**. Claude reads them once, distils a compact digest (~300–500 tokens), and stores it locally. From then on, `/work` checks for new sessions in each feeder and only reads the delta — keeping the ongoing token cost very low.

Folder structure

Feeder digests live inside the current project root:

```
Pub_XXX/
  _feeders/
    Pub_YYY1_digest.md    # compact summary of YYY1, generated by Claude
    Pub_YYY2_digest.md    # compact summary of YYY2
    notes.md              # free-text notes added via /add-memory
  _feeders.json           # registry: which feeders, last-synced date & git ref
```

/family — register feeder projects AI once per link

Run once when you want to connect another project. Claude reads the feeder's handover, session log, and LaTeX abstract, then writes a structured digest.

```
/family Pub_YYY1 Pub_YYY2
```

Token cost: ~2–4k per feeder on first run. After that, only deltas are read.

/work (updated) — feeder auto-update on every session open AUTO

When `/work` opens a project, it reads `_feeders.json` and checks each feeder's `_ai_log.md` for new session entries since the last sync. Updates happen automatically — no prompting.

- **No new sessions:** silent — costs ~200 tokens per feeder.
- **New sessions found:** automatically reads only the new blocks, appends a delta to the digest, updates `_feeders.json`. Reports what changed.

This means a collaborator's work on a feeder project is picked up the next time you open the dependent project — with no manual action required.

/add-memory — lightweight surgical memory AI

For adding a specific file, code snippet, or free-text note without digesting a whole project.

```
/add-memory C:\...\Pub_YYY1\code\model.py "GP surrogate architecture"
/add-memory Pub_YYY1              # adds _ai_log.md only + registers for monitoring
/add-memory "YYY1 uses a 3-layer GP surrogate trained on PMIP ensemble data"
```

Design rationale — why not a full vector RAG?

A vector RAG (embeddings + semantic retrieval) was considered and deliberately rejected. The digest approach is a better fit here because:

- You know precisely how projects relate — semantic search adds no value over a directed query
- Queries like *"go study this section of YYY1 and see if it applies here"* are more precise than anything retrieval would surface
- A full RAG requires an embedding pipeline, vector DB, chunking strategy, and staleness management — significant overhead for a benefit that is not needed
- `/add-memory` covers the surgical case when you want to pull in something specific without a full digest

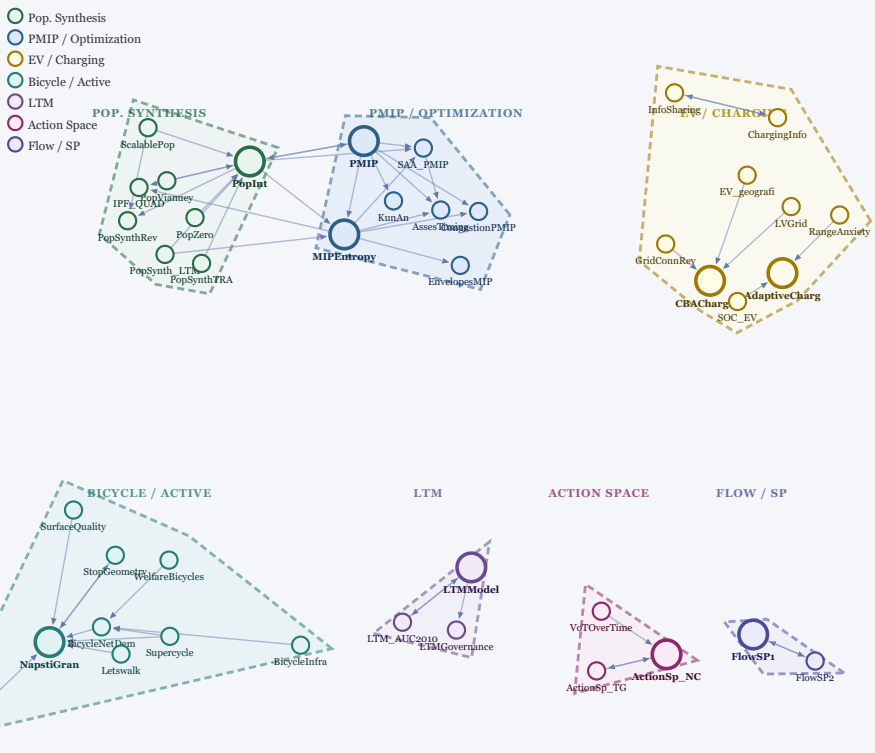
The summary-based model is intentionally simpler, cheaper, and sufficient.

Token cost summary

Operation	When	Approx. tokens
`/family` **(initial digest)**	Once per feeder link	~2–4k per feeder
`/work` **feeder check (no changes)**	Every session open	~200 per feeder
`/work` **feeder delta (new sessions)**	When feeder has new work	~500–2k per feeder
`/add-memory` **(file)**	Ad hoc	~500–1k

Current feeder network

Directed graph of all registered feeder relationships across active projects. An arrow A → B means A feeds knowledge into B. Larger outlined nodes are hubs (feed multiple consumers). Colour = research cluster. *Drag* nodes to explore; *scroll* to zoom.



E — Proxy-Sandbox: Per-Project File Isolation **AUTO on new_project**

Claude Code is not an OS-level sandbox — it runs with your full user account permissions. The **proxy-sandbox** design compensates for this through two complementary layers:

1. **Organisational:** sensitive files live *outside* the paper project folder. If it is not in the project, Claude will not see it.
2. **Technical:** a per-project `.claude/settings.json` deny list blocks Claude from reading known sensitive paths even if asked.

The result is a practical guardrail, not a cryptographic guarantee. The discipline of keeping data outside the project folder does the heavy lifting; the deny rules close the obvious escape hatches.

Shared sensitive data — `JR\Sensitive_Data\`

Many projects draw on the same sensitive source material (participant data, raw surveys, confidential datasets). Rather than duplicating or scattering this data, a single shared folder sits at the JR root — outside all project folders and outside Claude's reach by both layers of protection.

```
JR\
  Sensitive_Data\          <!-- human-only zone -->
    raw_data.csv
    participants.xlsx
    code\                  <!-- scripts you run yourself -->
      anonymise.py
  Publikationer\
    Pub_XXX\               <!-- Claude works here -->
    Pub_YYY\               <!-- Claude works here -->
  AI_auto\
```

Workflow: run scripts in `Sensitive_Data\code\` yourself → anonymised outputs land in `Pub_XXX\code\data\` → Claude picks up from there.

What is blocked

Deny rules operate at two levels:

Path	Scope	Reason
`JR/Sensitive_Data/**`	Global `~/.claude/settings.json`	Shared sensitive source data and analysis scripts
`JR/AI_auto/**`	Per-project `.claude/settings.json`	Infrastructure scripts, logs, API config
`AppData/**`	Per-project `.claude/settings.json`	Application credentials, browser data, tokens
`~/.claude/**`	Per-project `.claude/settings.json`	Global Claude config, memory files, session state

What is not blocked

Claude can still read files in the project folder and any other path not on the deny list. The primary protection against cross-project leakage is simply *not putting sensitive data inside a project folder*. If a file must be confidential, keep it in `Sensitive_Data\` or outside `JR\` entirely.

Generated file

```
Pub_XXX/
  .claude/
    settings.json    # auto-created by new_project.ps1
      deny: AI_auto/**, AppData/**, ~/.claude/**

~/.claude/settings.json   # global — deny: JR/Sensitive_Data/**
```

DTU data policy compliance

DTU's AI use policy states:

> *"You must avoid using personal or sensitive data in prompts regardless of the AI tool. You may use AI tools for which we do not have a data processing agreement, provided the content is not subject to GDPR or otherwise critical to the business."*

This setup satisfies that policy by construction:

Policy requirement	How it is met
No personal/sensitive data in prompts	Project folders contain only LaTeX source, research code, and non-personal data. Nothing personal is ever placed inside a project folder.
Confidential material → use a tool with DPA	Not applicable: no confidential material enters the project folder or the prompt.
Without DPA → only non-GDPR, non-business-critical content	All content Claude sees is academic research material: manuscripts, simulations, published data, code. Not subject to GDPR.

The proxy-sandbox enforces this not through trust alone, but structurally: Claude can only read what is inside the project folder, and the project folder is kept free of sensitive data by design. Compliance is therefore a property of the system, not a matter of remembering to be careful.

Design intent: Claude manages files responsibly, not by bypassing permissions. The proxy-sandbox makes accidental exposure hard — it does not make deliberate exposure impossible. Keep truly sensitive material outside the project root.

F — Typical Working Session

1. **Open project**: `helpi 5 Pub_XXX` — compiles LaTeX, then opens VS Code + Explorer + PDF in one step.
2. **Start Claude Code** in the project folder. Type `/work Pub_XXX` (or paste the folder path). Claude reads the log, checks feeder digests for updates, summarises where you left off, and asks for today's goal.
3. **Link feeder projects** (first time only): `/family Pub_YYY1` — Claude reads that project and stores a digest. Future sessions only check for deltas.
4. **Snapshot before risky edits** (optional): `/snapshot V2` git-tags the full `Overleaf_source/` state (.tex, .bib, figures, styles) before a major rewrite.
5. **Work** — Claude edits code or LaTeX. Each response auto-commits to git (Stop hook).
6. **Compile & preview** as needed: save in VS Code, or `helpi 6 Pub_XXX` from the terminal.
7. **Close the session**: `helpi 7 Pub_XXX` — runs `/close`, regenerates handover, and opens the browser in one step.
8. **Push to Overleaf** (`helpi 4 Pub_XXX`).
9. **Hand off to GPT**: run `helpi 7 Pub_XXX`, paste the markdown textarea into GPT. GPT adds its session block. Paste updated markdown back into `_ai_log.md`, rerun helpi 7.

G — Worked Example: `Pub_AssesTiming_Raoul_TBA`

Scenario. This paper studies the optimal timing of fleet electrification under uncertainty, using a Portfolio Mixed-Integer Programme (PMIP) with a Gaussian-process surrogate. It builds on two earlier projects: `Pub_PMIP_AOR` (the base PMIP fleet model) and `Pub_MIPEntropy_MPC` (entropy-regularised flexibility quantification). Collaborator Raoul also edits the LaTeX on Overleaf. Today's goal: run the full variance decomposition with the new GP surrogate and update the paper figures.

1 Check overall project status `helpi 13`

Before opening anything, get a bird's-eye view — which projects have uncommitted changes or are behind Overleaf.

```
PS> helpi 13 Project Status Dashboard — 2026-04-04 09:12
──────────────────────────────────────────────────────────── Pub_AssesTiming_Raoul_TBA last session 2026-03-
28 2 uncommitted Pub_PMIP_AOR last session 2026-03-15 clean Pub_MIPEntropy_MPC last session 2026-02-20 clean
Pub_PopInt_PartB last session 2026-01-10 clean ... (83 further projects)
──────────────────────────────────────────────────────────── 4 projects with recent activity | 2 with
uncommitted changes
```

2 Pull Overleaf changes `helpi 3`

Raoul may have edited the manuscript since the last session. Pull before touching anything locally.

```
PS> helpi 3 Pub_AssesTiming_Raoul_TBA [3] Pull one project from Overleaf -> Pub_AssesTiming_Raoul_TBA sync_one.ps1 -Project Pub_AssesTiming_Raoul_TBA [Enter] run | [any other key] cancel

Checking remote... changes found. Pulling Pub_AssesTiming_Raoul_TBA... done 1 file changed: main.tex (Raoul edited Section 3)
```

3 Open the project in Claude Code `/work`

Load context from the session log, check git status, and get a summary before starting work. Claude will ask for today's goal before writing anything to the log.

```
Claude> /work Pub_AssesTiming_Raoul_TBA Project : Pub_AssesTiming_Raoul_TBA Root : C:\...\Publikationer\Pub_AssesTiming_Raoul_TBA Last session — 2026-03-28 Agent : Claude Sonnet 4.6 Goal : Implement GP surrogate for PMIP Outcome : GP surrogate implemented; RMSE reduced from 0.12 to 0.04 Next : Run full variance decomposition with new surrogate; update figures in Section 4 Git status (code/): M strategic_pmip_killer/surrogate.py ?? results/gp_sweep_results.csv No feeders registered yet.

What is the goal for today's session? You> Run full variance decomposition sweep and update figures in Section 4.
Session block opened in _ai_log.md.
```

4 Link feeder projects *(one-time setup)* `/family`

This paper builds directly on the PMIP fleet model and the entropy-MIP flexibility work. Register them as feeders so their knowledge is available in every future session — and monitored for changes automatically.

```
Claude> /family Pub_PMIP_AOR Pub_MIPEntropy_MPC Reading Pub_PMIP_AOR... ✓ _handover.html found ✓ _ai_log.md (8 sessions, last: 2026-03-15) ✓ Overleaf_source/main.tex (abstract extracted) → Digest written: _feeders/Pub_PMIP_AOR_digest.md Reading Pub_MIPEntropy_MPC... ✓ _ai_log.md (5 sessions, last: 2026-02-20) ✗ _handover.html not found — using log + abstract only → Digest written: _feeders/Pub_MIPEntropy_MPC_digest.md _feeders.json updated. 2 feeders registered.

Pub_PMIP_AOR PMIP fleet model for EV adoption; stochastic scenario trees; key result: optimal switching threshold ~40% TCO parity. Pub_MIPEntropy_MPC Entropy-regularised MIP for flexibility; entropy term stabilises early-stage adoption curves under demand uncertainty. /work will now auto-check these at the start of each session.
```

5 Snapshot before risky edits `/snapshot` `helpi 8`

The variance decomposition refactor will touch `surrogate.py` heavily and regenerate all figures. Tag the current full manuscript state — including `.bib` and all figures — before anything changes.

```
Claude> /snapshot V1 Snapshot created Project : Pub_AssesTiming_Raoul_TBA Tag : snapshot-v1 Commit : 3fa19b2 Captured : 17 files (main.tex, refs.bib, 11 figures, sty, cls) Diff vs now : git -C "...\Overleaf_source" diff snapshot-v1 HEAD Restore file : git -C "...\Overleaf_source" checkout snapshot-v1 -- main.tex
```

6 Work — Claude edits code and LaTeX

Claude refactors `surrogate.py` to run the full variance decomposition sweep, regenerates `variance_decomposition_compare.png`, and updates the figure reference in Section 4. Each Claude response is auto-committed to git by the Stop hook.

```
[Claude edits strategic_pmip_killer/surrogate.py] [Claude runs the sweep — results written to results/gp_sweep_results.csv] [Claude regenerates pictures/variance_decomposition_compare.png] [Claude updates Section 4 caption in main.tex]
```

```
Auto-commit: claude: auto-commit 10:47 ✓ 3 files Auto-commit: claude: auto-commit 11:02 ✓ 2 files
```

7 Compile LaTeX and check the PDF `helpi 6`

Verify that the updated figure and caption compile cleanly before pushing anything to Overleaf.

```
PS> helpi 6 Pub_AssesTiming_Raoul_TBA [6] Compile LaTeX + open PDF -> Pub_AssesTiming_Raoul_TBA compile_latex.ps1 -
Project Pub_AssesTiming_Raoul_TBA [Enter] run | [any other key] cancel

Running latexmk (pdfLaTeX + BibTeX)... This is pdfTeX, Version 3.141592653 (MiKTeX 25.12) (./main.tex ...
./refs.bib ... 2 passes) Output written on main.pdf (19 pages, 1.4 MB) Compilation successful. Opening PDF...
```

8 Close the session `/close`

Log what was done, which files were touched, and what needs to happen next. Claude also auto-updates .claude/CLAUDE.md if the project state changed, and regenerates the handover — no extra steps needed.

```
Claude> /close Files touched (suggested — confirm or edit): ✓ code/strategic_pmip_killer/surrogate.py ✓
Overleaf_source/pictures/variance_decomposition_compare.png ✓ Overleaf_source/main.tex (Section 4 caption) Any
others? (Enter to skip)

One-sentence outcome: You> Full variance decomposition sweep completed; figure updated in Section 4. Next steps:
You> Run robustness checks across epsilon values; draft discussion section. Git ref: a4c8d91 Session block written
to _ai_log.md.

CLAUDE.md unchanged. Handover regenerated: _handover.html

Push to Overleaf if ready: helpi 4 Pub_AssesTiming_Raoul_TBA
```

9 Push edits to Overleaf `helpi 4`

Send the updated figure and LaTeX back to Overleaf so Raoul can see the changes and compile on his end.

```
PS> helpi 4 Pub_AssesTiming_Raoul_TBA [4] Push local edits to Overleaf -> Pub_AssesTiming_Raoul_TBA
push_to_overleaf.ps1 -Project Pub_AssesTiming_Raoul_TBA [Enter] run | [any other key] cancel

Staged changes: M main.tex M pictures/variance_decomposition_compare.png Committing: "Local edit 2026-04-04 11:18"
Pushing to Overleaf... To https://git.overleaf.com/... 3fa19b2..a4c8d91 master -> master Done. Overleaf will
reflect changes within a few seconds.
```

11 View project network graph `helpi 14`

See the full knowledge graph — all projects, feeder links, and activity. Useful to spot stale connections or find related projects you may have forgotten about.

```
PS> helpi 14 [14] Open project network graph network.ps1 [Enter] run | [any other key] cancel

Network graph generated Projects : 87 Feeder links: 2 Output : C:\...\AI_auto\network.html [browser opens —
Pub_AssesTiming_Raoul_TBA shown as diamond node with arrows from Pub_PMIP_AOR and Pub_MIPEntropy_MPC]
```

Next session — feeder auto-update in action

Two days later, Raoul has been working on `Pub_PMIP_AOR`. When you open `Pub_AssesTiming_Raoul_TBA`, the feeder digest is updated automatically — no manual action required.

```
Claude> /work Pub_AssesTiming_Raoul_TBA Project : Pub_AssesTiming_Raoul_TBA Last session — 2026-04-04 Goal : Run full variance decomposition sweep and update figures Outcome : Full sweep completed; figure updated in Section 4 Next : Robustness checks across epsilon values; draft discussion Git status (code/): clean

Feeders: Pub_PMIP_AOR 2 new sessions since 2026-03-15 — updating digest... Session 2026-04-05: extended scenario tree to 5 stages; new results in Table 3 Session 2026-04-06: sensitivity analysis on discount rate; findings affect switching threshold → _feeders/Pub_PMIP_AOR_digest.md updated. Pub_MIPEntropy_MPC up to date

What is the goal for today's session?
```

The new PMIP_AOR findings (extended scenario tree, revised switching threshold) are now part of the project's working knowledge — available to Claude without re-reading the full feeder project.

PART 3 — PEER AI AGENTS & COLLABORATION (H–I)

H — Shared Agent Skills — Codex, Gemini CLI, and beyond

Claude Code, GPT Codex CLI, and Gemini CLI all run as terminal agents with full filesystem access. Research skills are stored in `~\.agents\skills\` — a shared directory that all three agents discover automatically. Writing a skill once makes it available everywhere. The file infrastructure (helpi, PowerShell scripts, `_ai_log.md`, `_handover.html`) is shared identically across all agents.

Shared skill directory

```
~\.agents\skills\
  work\            SKILL.md + agents\openai.yaml   # alias for research-work
  close\           SKILL.md + agents\openai.yaml   # alias for research-close
  snapshot\        SKILL.md + agents\openai.yaml   # alias for research-snapshot
  family\          SKILL.md + agents\openai.yaml   # alias for research-family
  research-work\   SKILL.md + agents\openai.yaml   # full name (Codex fallback)
  research-close\  SKILL.md + agents\openai.yaml
  research-snapshot\
  research-family\
  grill-paper\     SKILL.md + agents\openai.yaml   # pre-submission stress-test
  grill-edit\      SKILL.md + agents\openai.yaml   # edit phase after grill-paper
  grill-me\        (Matt Pocock) general idea stress-test
  prototype\       (Matt Pocock) throwaway prototype
  tdd\             (Matt Pocock) test-driven loop
  diagnose\        (Matt Pocock) bug/regression diagnosis
  zoom-out\        (Matt Pocock) big-picture review
  caveman\         (Matt Pocock) ultra-compressed communication
  ... (14 Matt Pocock skills total)
```

Claude Code also symlinks skills to `~\.claude\commands\` so they appear as slash commands. Codex uses `agents\openai.yaml` metadata. Gemini CLI reads `SKILL.md` directly. Install new skills from GitHub with: `npx skills add <repo> --skill=<name> -y -g`

Research skills reference

Skill	Invocation	What it does	Claude equivalent
`work` / `research-work`	`/work Pub_XXX`, pasted project path	Reads `.claude/CLAUDE.md`, `_ai_log.md`, feeder digests, git status — asks for goal — writes session-start block	`/work`
`close` / `research-close`	`/close`	Writes close block to `_ai_log.md`, compresses log, regenerates handover, reminds to push to Overleaf	`/close`
`snapshot` / `research-snapshot`	`/snapshot V2`	Creates annotated git tag on `Overleaf_source/`	`/snapshot`
`family` / `research-family`	`/family Pub_YYY`	Reads feeder log + handover + abstract — writes digest to `_feeders/` — updates `_feeders.json`	`/family`
`grill-paper`	`/grill-paper`	Pre-submission stress-test. AI reads manuscript, runs three blocks (positioning,	`/grill-paper`

		rigor, freestyle), grades each answer 0–10 on paper quality and author understanding, logs all issues to `Overleaf_source/grill_log.md`. No edits to manuscript during session. Log persists across sessions; issues raised in multiple sessions are flagged as strong signals.	
`grill-edit`	`/grill-edit`	Edit phase after `/grill-paper`. Works through master issue list (blocking → significant → minor), proposes concrete edits, records author decisions (approve / adjust / dismiss with reason), updates verdict to READY when all blocking issues resolved.	`/grill-edit`
`pipeline`	`/pipeline "task" --project Pub_XXX` `/pipeline "task" --agents gemini,claude,codex,claude`	Background multi-agent job. Chains Claude → Gemini → Codex → Claude (configurable via `--agents`). Each agent receives the task plus all prior output. Runs as a detached PowerShell process — you leave, it runs. On completion: `final.md` opens, a headless Claude subprocess writes a session block to `_ai_log.md`, and calls `helpi 7` to regenerate the handover. Output lands in `AI_auto\_pipelines\YYYY-MM-DD_HH-MM-SS\`. Treats the full run as one session with auto-close. Requires `--project` for log and handover update; without it, produces files only.	`/pipeline` (Claude-only skill; also usable via `claude -p` for Codex/Gemini calls within the pipeline)

Gemini CLI

Gemini CLI (`gemini`, v0.42+, Google AI Pro) is a third peer agent alongside Claude Code and Codex. Global instructions live in `~\.gemini\AGENTS.md`. Per-project context is loaded via `AGENTS.md` in the project root (auto-generated by `helpi 7`) — Gemini reads this automatically at session start via `context.fileName: "AGENTS.md"` in `~\.gemini\settings.json`. No per-project setup needed.

Shared session log

All agents write identical `## Session YYYY-MM-DD` blocks to `_ai_log.md`. Sessions from all agents interleave cleanly — the **Agent:** field records who ran each session (e.g. *Claude Sonnet 4.6*, *GPT-5.4*, *Gemini CLI*). Switching agents mid-project is seamless in any direction.

Agent capability matrix

Capability	Claude Code	Codex CLI	Gemini CLI
Open/close session	✓	✓ via `/work` skill	✓ via `/work` skill
Snapshot, family, grill-paper, grill-edit	✓	✓ via skills	✓ via skills
Run helpi commands	✓	✓	✓ (use `! helpi N proj -Force`)
Read _ai_log.md and feeders	✓	✓	✓
Background pipeline (`/pipeline`)	✓ (orchestrates all agents)	— (runs as a round inside pipeline)	— (runs as a round inside pipeline)
Convergence Forum (`helpi 25`)	✓ (orchestrates/moderates)	✓ (participant)	✓ (participant)
Reviewer response loop (`/respond`)	✓	✗	✗
Submission package (`/submit`)	✓	✗	✗
Auto-commit after every response	✓ (Stop hook)	✗	✗
Global instructions	`~\.claude\CLAUDE.md`	`~\.codex\config.toml`	`~\.gemini\AGENTS.md`
Per-project context	`.claude/CLAUDE.md` (auto)	`AGENTS.md` (via skill)	`AGENTS.md` (auto)

Shared context — zero manual handover: `AGENTS.md` (auto-generated by `helpi 7` or `/close`) is loaded automatically by all three agents at session start. It contains the latest session snapshot and full log summary. All agents write to the same `_ai_log.md` and the same `grill_log.md` — switching mid-project or mid-grill is seamless.

Installing the Infrastructure

Two scenarios: **new machine, files already present** (OneDrive synced your folders back) and **first-time install for a new colleague**. Both use the same two scripts.

Scenario A — Replacement machine (your files are intact via OneDrive)

Your `AI_auto\` folder and all project folders are already synced. You just need to re-wire the tooling:

```
.\scripts\restore.ps1        # or: helpi 20
```

This runs an 8-step checker and fixes what it can automatically:

1. Claude Code CLI present and logged in
2. Git installed
3. MiKTeX installed
4. VS Code installed
5. PowerShell profile — adds `helpi` function if missing
6. `~\.claude\` folder — checks `CLAUDE.md` and `commands\` are present
7. Scheduled task `AI_AutoSync` — re-registers if missing
8. `projects.json` — confirms it is present (all project registrations travel with OneDrive)

Fix any `[ERR]` items it reports, restart your terminal, and you are back.

Scenario B — New colleague (clean machine, no files yet)

Clone the `AI_auto` repo, then run the setup wizard:

```
git clone https://github.com/jepperich69/ai-infrastructure AI_auto
cd AI_auto
.\scripts\setup.ps1          # or: helpi 21
```

The wizard walks through 7 steps:

1. Publications root folder (where all `Pub_*` / `Pro_*` / `PhD_*` folders live)
2. Git identity (name + email)
3. Tool path detection (VS Code, MiKTeX — prompts if not found at default locations)
4. Writes `scripts\config.local.ps1` with all machine-specific paths (this file is gitignored — never causes merge conflicts)
5. Wires `helpi` into the PowerShell profile
6. Registers the `AI_AutoSync` scheduled task (auto-pull every 4h)
7. Offers to run the Overleaf bulk-import (see below)

config.local.ps1 — machine-specific settings

Since v1.0, machine-specific paths (publications root, VS Code path, MiKTeX bin, etc.) live in `scripts\config.local.ps1`, which is **gitignored**. The shared `scripts\config.ps1` contains no personal paths — it just loads `config.local.ps1` at runtime.

This means `git pull` (helpi 26) can never conflict with your local settings. If you need to set up a new machine manually, copy `scripts\config.local.example.ps1` to `scripts\config.local.ps1` and fill in your paths.

Connecting Overleaf projects — two approaches

Option 1: Bulk import (recommended for ≥3 projects)

Fetches all your Overleaf projects in one go using your browser session cookie:

1. Log into `overleaf.com` in your browser
2. Press `F12` → Application → Cookies → copy the value of `overleaf_session2`
3. Run: `.\scripts\fetch_overleaf_projects.ps1` — paste the cookie when prompted
4. Review the printed table. Each Overleaf project is fuzzy-matched to a local folder. Fix any `??? NO MATCH` rows in the exported `overleaf_projects.csv`.
5. Run: `.\scripts\link_projects.ps1` — clones every matched project into `Overleaf_source\` and registers it in `projects.json`

The setup wizard (step 7) offers to run this inline so you can finish fully wired in one session.

Option 2: One project at a time (fine for getting started)
Register each project as you need it. The Overleaf Git URL is under **Menu → Git** in any Overleaf project:

```
helpi 1 Pub_MyPaper https://git.overleaf.com/abcdef123456
```

This clones the project into `Publikationer\Pub_MyPaper\Overleaf_source\` and adds it to `projects.json`. From that point:

Command	What it does
`helpi 2 Pub_MyPaper`	Push local edits → Overleaf (colleagues see them immediately)
`helpi 3 Pub_MyPaper`	Pull Overleaf → local (get edits made on Overleaf or by colleagues)
auto every 4h	Scheduled task runs helpi 3 for all registered projects silently

Where does GitHub fit? The `AI_auto\` scripts repo lives on GitHub — that is just for versioning the infrastructure itself. Paper content lives only in the Overleaf Git remote. There is no separate GitHub repo per paper.

Keeping the infrastructure up to date *MANUAL*

Once installed, pull the latest scripts and fixes from GitHub with:

```
helpi 26
```

This runs `scripts\update.ps1`, which:

1. Checks you have a git remote configured
2. Warns you of any uncommitted local changes (does not overwrite them)
3. Runs `git pull origin master`
4. Automatically fixes Claude Code hook paths in `~\.claude\settings.json` if they still point to the old pre-v1.0 root location

Your `scripts\config.local.ps1` is gitignored and is **never** touched by a pull. After a pull, check `CHANGELOG.md` to see what changed.

Prerequisites (install manually before running setup)

Tool	Where to get it
Claude Code CLI	`claude.ai/code` — then run `claude login`
Git	`git-scm.com/download/win`
MiKTeX	`miktex.org/download`
VS Code	`code.visualstudio.com`
Strawberry Perl	`strawberryperl.com` — needed for `latexdiff` in `submit.ps1`
SymPy (Python)	`& "C:\Users\rich\AppData\Local\miniconda3\python.exe" -m pip install sympy` — required for `/verify-math` (machine math verification). Without it the skill stops and asks you to install it.

Optional capability: SymPy is not needed for the core write/sync/compile workflow, but `/verify-math` will not run without it. Install it once into the miniconda `base` env to unlock machine-checked algebra, series, solve-inversions, moments, and numeric

calibrations. See A3 — /verify-math.

I — Collaborative Setup

This infrastructure is designed for a single researcher, but it extends naturally to collaborative work. The key insight is that the infrastructure splits into two layers with very different portability: one layer lives in the git repo and travels for free; the other is personal and must be installed once per machine per person.

What travels with git (zero setup for collaborators)

These files are committed to each project repo and are immediately available to any collaborator who clones or pulls the project:

- `.claude/CLAUDE.md` — project brief, paper description, co-authors, key files, constraints
- `_ai_log.md` — full session history from all contributors and agents
- `_handover.html` (or `AGENTS.md`) — generated context snapshot, auto-read by Claude and Codex at session start
- `_feeders/` and `_feeders.json` — digests of related projects (if registered)

Any collaborator with Claude Code installed can open the project with `/work Pub_XXX` and is immediately oriented — no re-briefing, no manual handover. The session log shows who did what and when; the handover shows where the project stands right now.

Personal infrastructure (each person installs once)

The file-layer automation is not in the project repo — it lives on each person's machine:

Component	What to do
Claude Code	Install from `claude.ai/code` and authenticate with a personal account
Global `CLAUDE.md`	Copy or adapt `~\.claude\CLAUDE.md` — sets research paths, session discipline, logging rules
`AI_auto\` **scripts**	Clone or copy the `AI_auto` folder to their machine; add a `helpi` alias pointing to their local copy
Overleaf git bridge	Each person sets up the Overleaf remote on their local clone of the project (paid Overleaf plan required)
GPT Codex CLI (optional)	Install skills from `~\.codex\skills\` if using Codex as an alternative agent

Minimum viable setup for a collaborator: Install Claude Code, copy the global `CLAUDE.md`, clone the project repo. That is enough to open sessions with `/work`, have full project context, and write to the shared log. The `helpi` file scripts are optional — they automate Overleaf sync and PDF compilation but are not needed for AI-assisted writing.

Branch discipline

Each person works on their own branch. The AI agent works on that branch too. Coordination happens through git — not through AI agents talking to each other (they have no shared working memory).

- Assign ownership by section or task, not by time slot — avoids merge conflicts in `.tex` files
- Use short-lived branches named by contributor and section: `jeppe/methods`, `coauthor/results`
- Run `helpi 4` (rebase push) before opening a PR — it handles diverged Overleaf history automatically
- The project lead merges to `main` and pushes to Overleaf; collaborators pull via `helpi 3`

Shared session log across contributors

`_ai_log.md` is the single source of truth for what happened and when. Each session block includes an **Agent:** field and implicitly the git author name — so the log naturally shows interleaved work from multiple people and agents:

```
## Session 2026-04-18  [Jeppe — Claude Sonnet 4.6]
Goal: Rewrite methods section intro
...

## Session 2026-04-19  [Co-author — GPT-5.4]
```

```
Goal: Add robustness checks to results
...
```

Neither person edits or deletes the other's entries. The next session by either party reads the full log and picks up exactly where the other left off.

Per-person handovers via Overleaf

Each infrastructure user produces a `_handover_[initials].md` in `Overleaf_source/` at every `/close`. These files sync to Overleaf with the next `helpi 2` push. When you start a session on a paper a collaborator worked on the night before, `/work` reads their handover file and surfaces it at session start — goal, outcome, next steps — without any manual coordination. Collaborators who do not use the infrastructure produce no such file; `/work` skips silently.

Overleaf co-authoring

Overleaf remains the shared editing surface for collaborators who do not use the local infrastructure. The git bridge means edits from Overleaf (by co-authors) flow into the git history via `helpi 3` (pull), and local AI-assisted edits push back via `helpi 4`. The rebase-on-push strategy in `helpi 4` handles diverged histories cleanly in most cases; if the same lines were edited on both sides, standard git conflict markers appear in the `.tex` file for manual resolution.

Co-authors who only use Overleaf need no infrastructure at all — they just write in Overleaf as normal. The infrastructure user pulls their changes in, works with Claude, and pushes back.

J — Environment Map & Known Issues

A compact *Platform facts* block is embedded at the top of `~/.claude/CLAUDE.md` and `~/.codex/config.toml` so every agent session starts with current knowledge of this machine — no probing, no rediscovery. The full reference lives in `AI_auto/known_issues.md`.

Software on this machine

Tool	Version	In PATH?	Full path (if not in PATH)
PowerShell	7.6.1	Yes	—
R	4.5.2	**No**	`C:\Users\rich\AppData\Local\Programs\R\R-4.5.2\bin\R.exe`
Python (miniconda)	3.13.9	**No**	`C:\Users\rich\AppData\Local\miniconda3\python.exe`
LaTeX (MiKTeX)	—	Yes	`pdflatex`, `latexmk`, `xelatex` all work
git	2.54.0	Yes	—
gh (GitHub CLI)	2.88.1	Yes	—
Node.js	v24	Yes	—
Stata	—	—	Not installed
Julia	—	—	Not installed

Conda environments: `base` (default), `pyopt`. SymPy 1.14.0 is installed in `base` — required by `/verify-math`; install with `& "C:\Users\rich\AppData\Local\miniconda3\python.exe" -m pip install sympy`.

Key platform rules

- `python3` is a Windows Store alias — returns exit 49. Always use the full miniconda path.
- Use the **PowerShell tool** (not Bash) for any non-trivial PS command — quoting breaks inside Bash.
- After writing or editing any `.ps1` file, check for curly quotes and fix them.
- Paths contain spaces (`OneDrive - Danmarks Tekniske Universitet`) — always double-quote.
- Never use `pwsh -EncodedCommand` inline — write a `.ps1` file and call with `-File`.

Map maintenance

At `/close`, Claude automatically checks whether any new platform fact was discovered this session and adds it to both the inline block and `known_issues.md` (1–2 entries max; one-off flukes ignored). The map grows without manual upkeep.

Infrastructure Versioning

The infrastructure itself is versioned so that you can recover a working state, understand what changed between sessions, and avoid accidentally breaking things mid-project. The current version is **v1.0** (see `VERSION` file — single source of truth).

Workflow in plain language

- **Stable line (`main`)** — what you rely on day-to-day. Each release is permanently marked with a tag like `v0.1`.
- **Development line (`develop`)** — where larger ongoing changes accumulate before they are tested and promoted. For small fixes, committing directly to `main` is fine.
- **Promoting a release** — update `CHANGELOG.md`, bump `VERSION`, update the version field in this file, then `git tag v0.X`.

When does a change deserve a new version?

- A new `helpi` command or slash command is added or removed
- An existing command's behavior changes in a way that would break current usage
- A safety or reliability improvement that affects the normal workflow
- A structural change to how projects are stored or registered

Minor tweaks — wording in docs, small bug fixes that don't change behavior — do not need a version bump.

Key files

File	Purpose
`VERSION`	Current version number at a glance
`CHANGELOG.md`	What changed in each version and what is coming next
`RELEASING.md`	Step-by-step instructions for making a new release

NoteTaker: Mobile Voice Bridge

A serverless bridge between mobile devices and the research infrastructure.

- **Phone App:** GitHub Pages (jepperich69/notetaker-app)
- **Proxy:** Supabase Edge Function (dpslhidnmifmbmrhugea)
- **Spool:** GitHub Releases (jepperich69/notetaker-inbox)
- **PC Engine:** code/notetaker_watcher.ps1 (Resident Mode)